Choosing the Right Kernel

How SVM Decision Boundaries Change with Data Geometry

A practical deep dive into Linear, RBF, and Polynomial kernels on the Steel Plates Faults dataset

Muhammad Usama Akram | Student ID: 24178272 | GitHub

Introduction

Real industrial datasets rarely exhibit perfect class separability; therefore, SVMs seek the hyperplane that maximizes the margin between classes in the original feature space. This is achieved implicitly by kernels, employing a technique called the kernel trick, which transforms data into a higher-dimensional space, allowing for linear separation in that new space. The selection of the kernel is not purely aesthetic, since it dictates the overall shape of the decision boundary, and the wrong kernel selection can result in a difference of 26 points (50% vs. 76%).

One question this tutorial attempts to answer is: how does the choice of kernel affect the SVM's learning on a real dataset that has overlapping classes and imbalanced classes, and what kernel(s) will fail when? Our testbed is the UCI Steel Plates Faults dataset (1941 samples, 27 features, 7 fault types), which is a true manufacturing quality-control problem, much more representative of real datasets.

The Dataset

The Steel Plates Faults dataset was captured by the Semeion Research Center, Italy, and was an actual industrial classification problem: automatically classify the type of surface defect on a stainless steel plate, based on 27 geometric and luminosity measurements obtained by an automatic inspection system. There are 7 distinct fault categories: Pastry, Z_Scratch, K_Scatch, Stains, Dirtiness, Bumps, and Other_Faults; each sample is associated with a single fault category.

Property	Value
Samples	1,941
Features	27 (geometric shape descriptors: area, perimeter, luminosity, edge indices, etc.)
Classes	7 fault types (Pastry, Z_Scratch, K_Scatch, Stains, Dirtiness, Bumps, Other_Faults)
Source	UCI ML Repository (ID: 198)

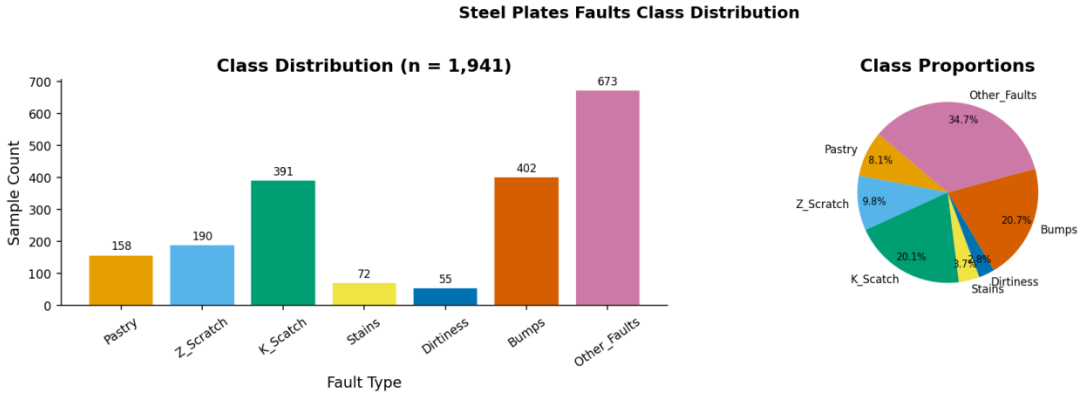

Figure 1: Class distribution of the Steel Plates Faults dataset.

It's immediately apparent from Figure 1 that the data is highly imbalanced. Other_Faults constitutes 34.7% of the samples and Dirtiness 2.8%. This imbalance will impact each kernel differently, and we're not only interested in overall accuracy, but we're also interested in per-class F1 scores as well. Furthermore, the numerical range of the 27 features is also widely varying; thus, it is essential to standardize them before using any algorithm that relies on distance, like SVM.

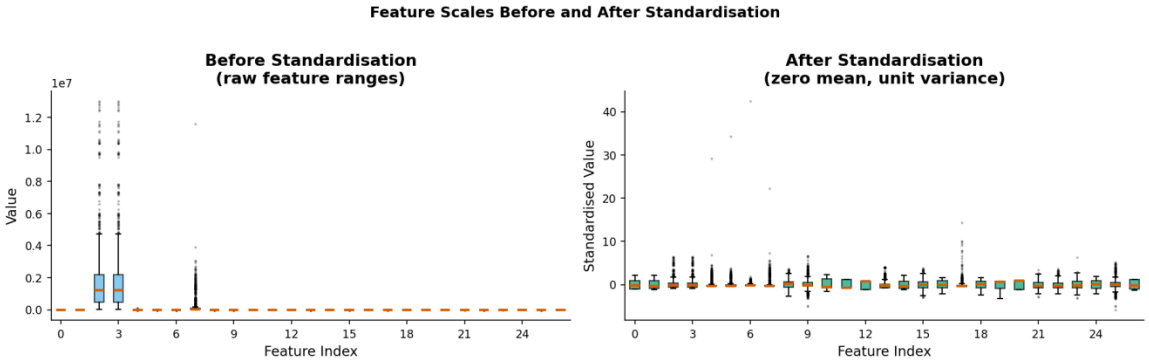

Figure 2: Feature scales before and after standardization.

Some of the raw features are as high as 10^7, others are around zero, as indicated in Figure 2. If the features were not scaled, those with large magnitude would dominate the computation of the kernel completely. Before any model is trained, zero-mean and unit variance transformations (clipping outliers to fit into the distribution of the data) are performed on all features, and are fitted only on the training set so as not to leak any data.

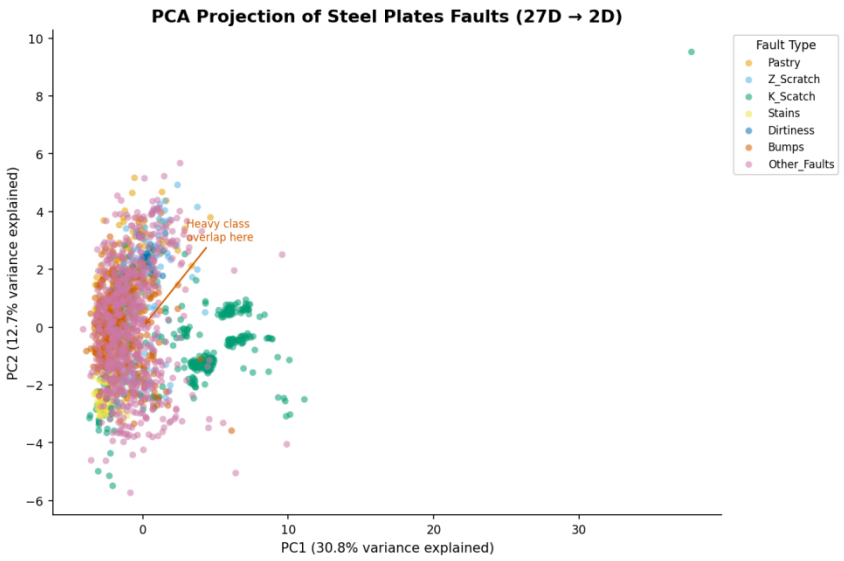

Figure 3: PCA projection from 27D to 2D (PC1=30.8%, PC2=12.7% variance explained).

The main driving force for this tutorial is the PCA projection in Figure 3. Just 43.5% of the variance is accounted for by the two dimensions, validating the high-dimensional nature of the data. The central mass of the overlaps is immediately apparent and indicates that a linear boundary will not be enough; thus, non-linear kernels will be required.

SVM Theory: From Hyperplanes to Kernels

The Core Idea

An SVM computes the hyper-plane w^Tx + b = 0 which has the largest margin between the closest points of the two classes (support vectors). The optimization problem is: minimize $\frac{1}{2} \| w \|^2$subject to $y_i(w^T x_i + b) \geq 1$. A critical property of this elegant formulation is that only the support vectors are used to define the boundary - this makes SVMs memory efficient and resistant to points far from the margin. The value of the regularization parameter C balances the bias-variance dilemma. A high C will be severely punitive for misclassification and may give a very small margin, which could be overfit. A low C allows more mistakes, but has a broader, more generalizable margin.

The Kernel Trick

When data is not linearly separable in the input space, but we implicitly map to a higher-dimensional feature space using a kernel function, $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$. The inner products can be calculated without ever evaluating the function phi explicitly and are therefore, in the case of the RBF kernel, numerically feasible even for an infinite-dimensional feature space.

The Three Kernels Compared

Kernel	Key Parameters	Best Suited For
Linear	C	High-dimensional, linearly separable data
RBF (Gaussian)	C, gamma	Radially clustered, complex non-linear boundaries
Polynomial	C, gamma, degree	Data with known polynomial feature interactions

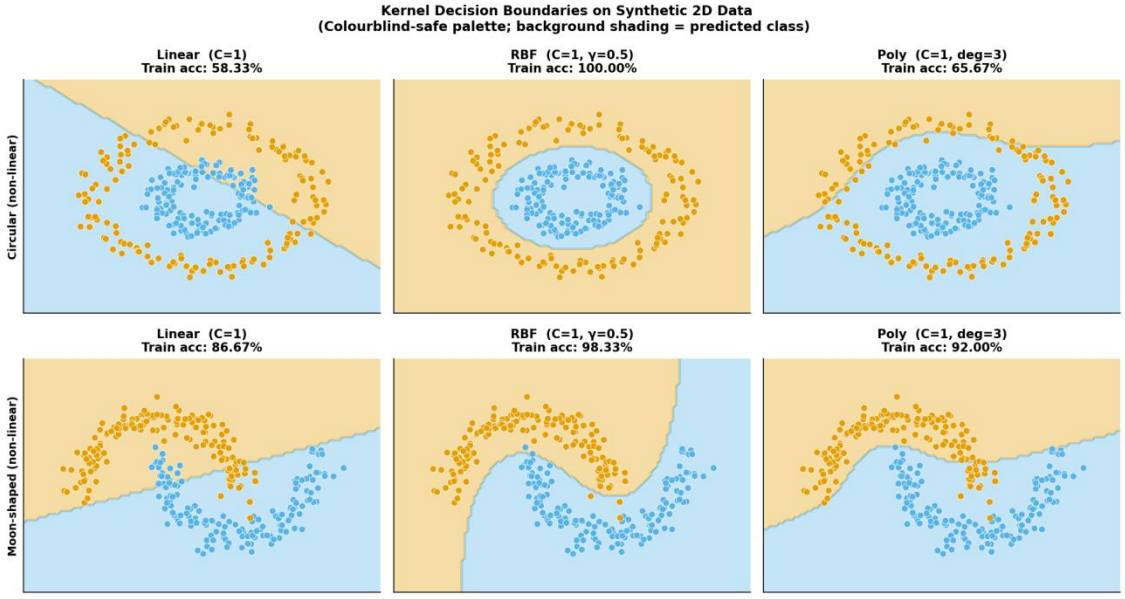

Figure 4: Kernel decision boundaries on synthetic 2D data

The contrast in Figure 4 is in stark relief. The linear kernel is one straight cut through space. The RBF kernel is a very flexible kernel around the clusters. The polynomial kernel is more adaptive than linear, but less flexible than the RBF kernel for purely radial structures, and is situated between them.

Kernel Comparison on Steel Plates Faults

Decision Boundaries

The three kernels were trained with 80% of the data (1,553 samples, stratified by class) with the 27 standardized features. The decision boundaries are illustrated in Figure 5 on the first two PCA components.

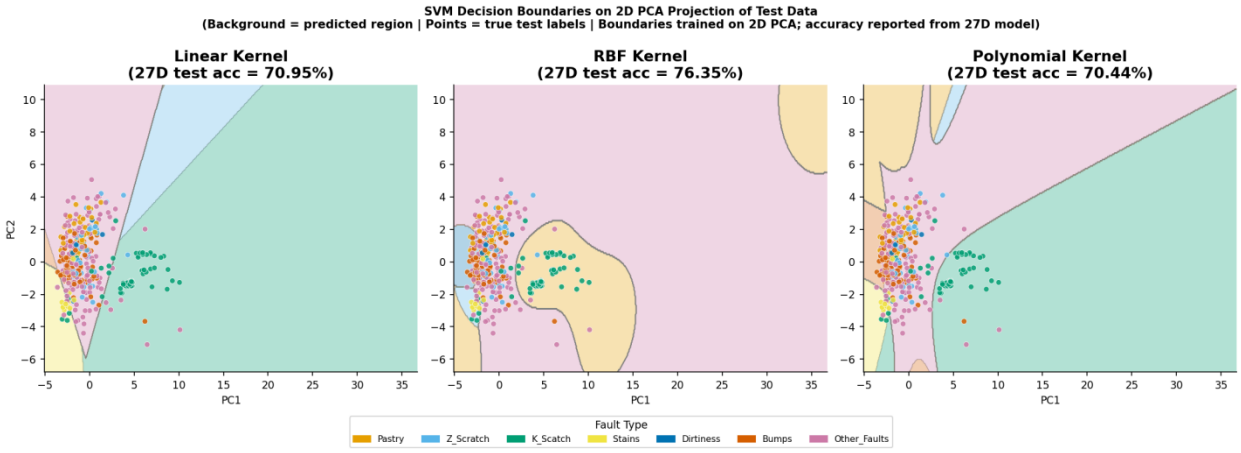

Figure 5: SVM decision boundaries on the 2D PCA projection

The basic geometry of the kernels is shown in the boundary shapes in Figure 5. The linear kernel divides space with straight lines, which are sufficient for well-separated classes, but do not work well for curved boundaries. The RBF kernel has a clear indication of the bend around the K_Scatch cluster. The boundaries are definitely irregular when the polynomial kernel is used..

Per-Class Performance

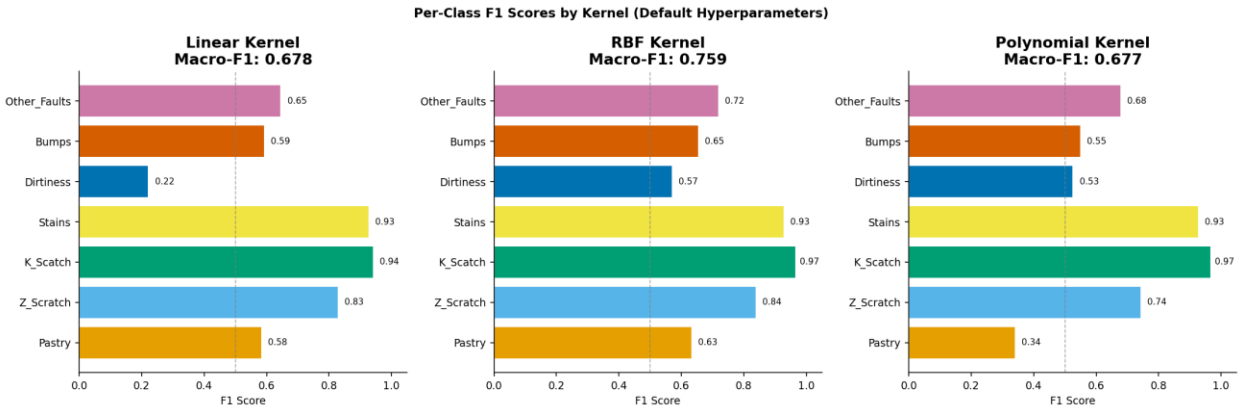

Figure 6: Per-class F1 scores for all three kernels.

The most educational figure in this tutorial is Figure 6. The overall accuracy hides the real numbers: the linear kernel has an F1=0.22 for the rarest class (Dirtiness, n=55), whereas the RBF kernel has an F1=0.57, which is more than double. This difference is directly related to the kernel's learning ability in the small, tight cluster, which Dirtiness creates in the feature space.

Hyperparameter Sensitivity: C and Gamma

Selecting a kernel is just half the battle. In this dataset, the regularization parameter C and the kernel-width parameter gamma (in the case of RBF and Polynomial) vary the accuracy by more than 30 percentage points.

Effect of C Across All Kernels

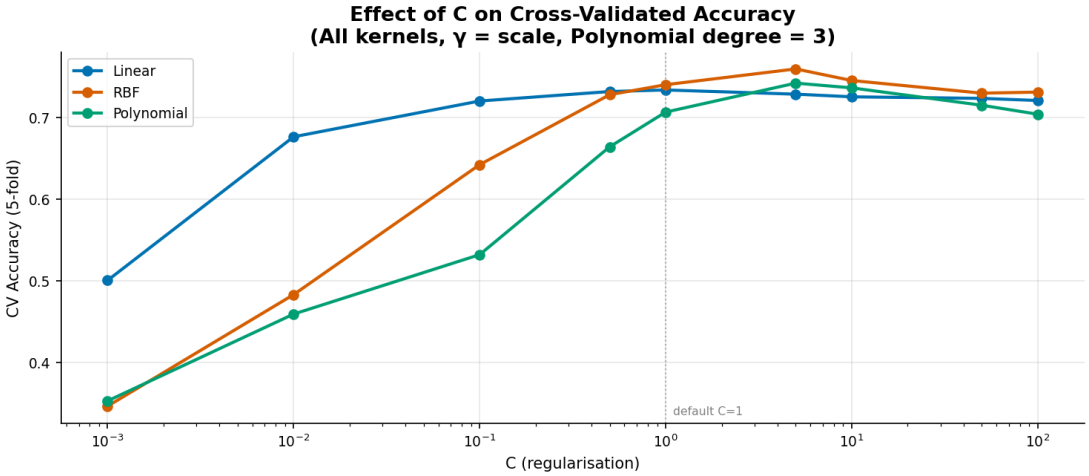

Figure 7: Cross-validated accuracy (5-fold) as C varies across three orders of magnitude.

Figure 7 demonstrates that C=1 (the default) is actually close to optimal for Linear and RBF on this dataset. The Polynomial kernel's sensitivity to C is notable: it lags badly at low C values, suggesting its implicit feature space requires more aggressive margin control to be useful.

RBF Kernel: The C x Gamma Interaction

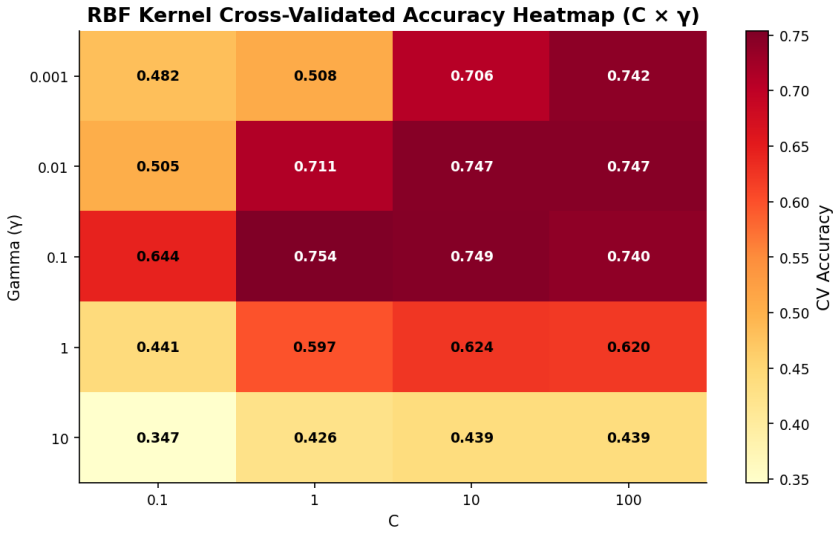

Figure 8: RBF kernel cross-validated accuracy heatmap across C and gamma values

Figure 8 is a crucial practical tool, the C * gamma heatmap. A high gamma means that each training point has only a very small influence on the boundary, so that it memorizes the training set and not generalizes. Optimum configuration (C=1, gamma=0.1) gets an accuracy of CV = 0.754.

Polynomial Kernel: Degree Sensitivity

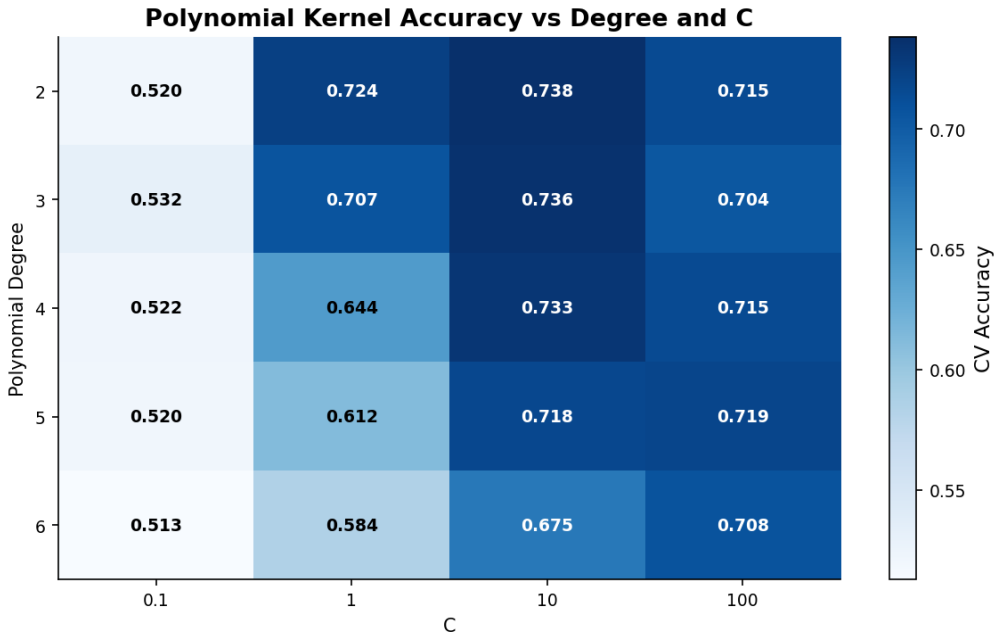

Figure 9: Polynomial kernel accuracy across degree and C.

The interesting and unexpected thing in Figure 9 is that higher degree polynomial boundaries (complexity) do not necessarily lead to higher accuracy on this data. The results indicate that degree 2 outperforms degree 3 at the optimal C, indicating that the Steel Plates data has a lower-order polynomial structure.

When Each Kernel Fails

Failure modes are crucial, as well as understanding when the kernel is working. The two basic failure modes, underfitting and overfitting, are shown by deliberately using bad hyperparameter settings in Figure 10.

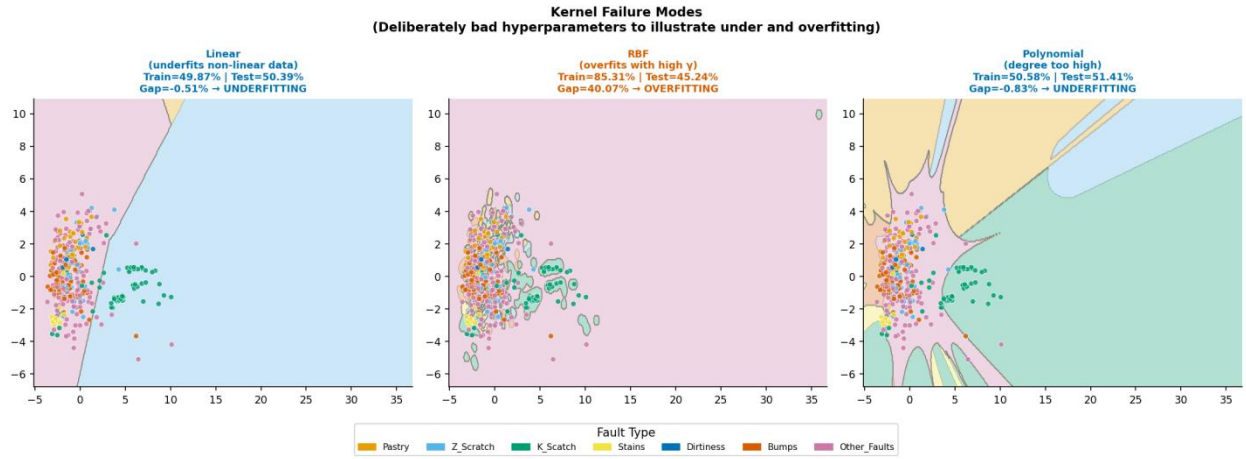

Figure 10: Kernel failure modes using deliberately poor hyperparameters

The case of overfitting from the RBF in Figure 10 is enlightening. The jagged islands in the middle of the RBF panel are the model memorizing individual training points, instead of learning the fault geometry. The gap between the train and test is 40.07%, which is a textbook example of overfitting.

Kernel	Failure Mode	Symptom	Root Cause
Linear	Underfitting	Low train & test acc	C too small
RBF	Overfitting	High train, low test acc	gamma too high
Polynomial	Instability	Erratic boundaries	Degree too high

Finding the Best Model with GridSearchCV

The grid search with stratified 5-fold cross-validation (CV) was employed for all three kernels and their primary hyperparameters. By wrapping the StandardScaler and SVC in a Pipeline, we ensure that scaling is fitted only to the training folds, thereby eliminating the risk of data leakage.

The confusion matrix in Figure 11 once again shows that RBF with C=1, gamma=0.1 is the optimal setting. For K_Scatch, we get almost perfect classification, matching the well-separated cluster in Figure 3. The Bumps -> Other_Faults confusion (22 samples) is a real geometric ambiguity as both classes have different geometries of faults that partly overlap in the 27D feature space.

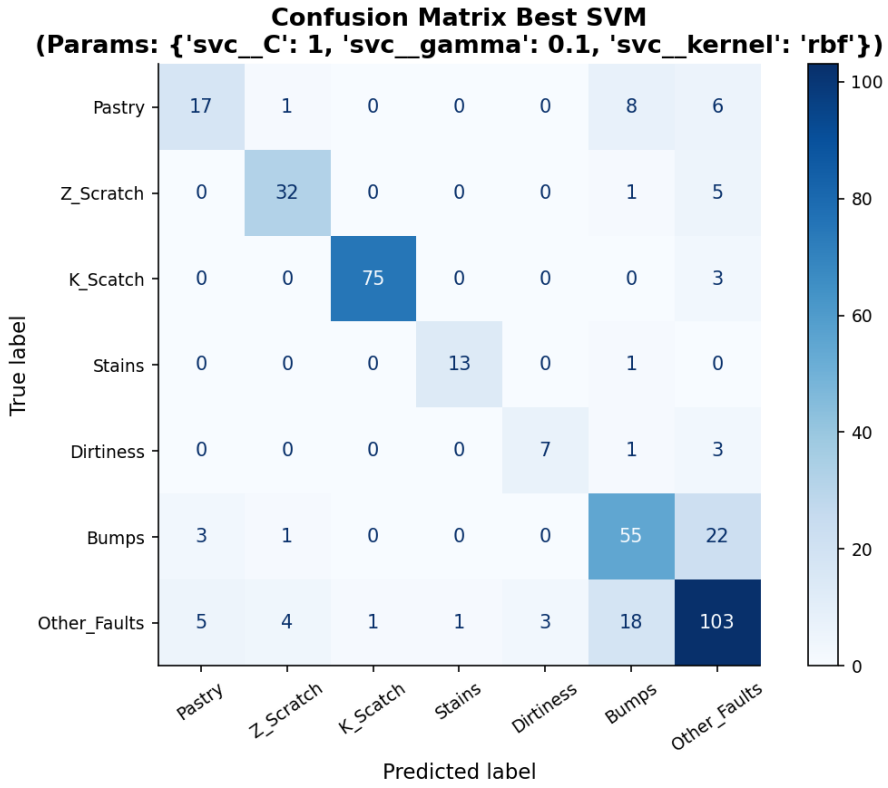

Figure 11: Confusion matrix for the best model: RBF kernel with C=1, gamma=0.1

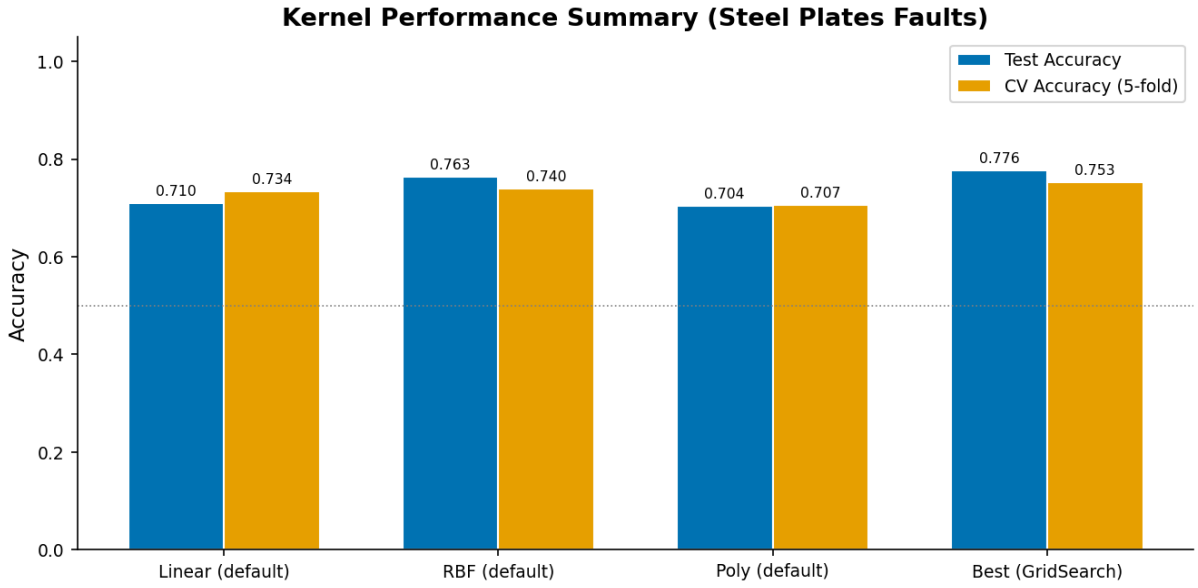

Figure 12: Performance summary across all kernels.

As the final word suggests in Figure 12 , at default and tuned RBF is the better kernel for this dataset, and the difference between the two (+1.3 percentage points) provides a confirmation that hyperparameter optimization is beneficial, but not game-changing when the kernel is already correct.

Practical Kernel Selection Guide

The following decision chart summarizes the use of each kernel, based on the experiments in this tutorial:

Kernel	Use	Avoid	Key Tuning	Speed
Linear	High-dimensional text/genomics data; known linear separability	Classes overlap in complex, curved patterns	C (try: 0.1 to 100 on log scale)	Fastest
RBF	Unknown structure; overlapping classes; default choice	γ is very high (overfits); large datasets (slow to train)	C and γ jointly (use heatmap search)	Moderate
Polynomial	Domain knowledge of polynomial feature interactions	Degree > 4; no known polynomial structure in data	Degree (2–4) and C	Slower at high degree

The recommendations for the Steel Plates dataset are as follows: to begin with, as a good starting point, use RBF (C=1, γ=0.1); if the performance of the minority classes is significant, consider using class_weight='balanced'; and never forget to standardise features before model training. Alternatively, LinearSVC (which utilizes a linear kernel optimized with liblinear) or kernel approximation can be used if training time is an issue, to achieve speedup with little loss of accuracy.

Ethical Considerations

Fairness and Class Imbalance

The Steel Plates dataset is imbalanced, with the ratio between the largest and smallest classes being 12:1. Optimizing for the overall accuracy would lead to the model basically disregarding rare fault types in an automatic quality control system. Class-weighted training (class_weight='balanced') or oversampling methods like SMOTE should be used for any deployment, and the evaluation should take place according to per-class recall and F1 scores and not only based on accuracy.

Transparency and Explainability

SVMs are not interpretable models, especially if using the RBF kernel. The decision boundary is in a kernel-induced feature space, which is not directly visualizable or understandable by an operator on the factory floor. For safety-critical deployment scenarios, it is advisable to use SVM predictions in conjunction with post-hoc explanation techniques like SHAP or LIME.

Dataset Provenance and Consent

The UCI Steel Plates Faults dataset is released with a Creative Commons BY 4.0 licence, which allows users to freely use it academically or commercially with attribution. Personally identifiable information is not included in the data. The user who uses this tutorial for proprietary manufacturing datasets should ensure data collection has taken place with the appropriate organization consent.

Environmental Cost

GridSearchCV with 5-fold cross-validation of 36 hyperparameter combinations would need 180 SVM models to be trained. This takes a few minutes on a commonly used laptop for the 1,941 samples in this data set. This method would be very cost-intensive, however, if it was applied to hundreds of thousands of samples. Random search or Bayesian optimization should be considered as alternatives to energy-intensive methods.

Responsible Deployment

The distribution of faults in the data collection period will be reflected in a model trained on historical fault data. Changes in the manufacturing process may lead to changes in the distribution of faults, which can result in a decrease in the performance of the model without any warning. Monitoring of distribution shift should be used in any production deployment, and there should be a retraining plan.

References

[1] Buscema, M., Terzi, S., & Tastle, W. (2010). Steel Plates Faults [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5J88N

[2] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297. https://doi.org/10.1007/BF00994018

[3] Schölkopf, B., & Smola, A. J. (2002). Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond. MIT Press.

[4] Scikit-learn developers (2024). sklearn.svm.SVC — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

[5] Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2016). A Practical Guide to Support Vector Classification. https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf

[6] Müller, A. C., & Guido, S. (2016). Introduction to Machine Learning with Python. O'Reilly Media. Chapter 2.

[7] Scikit-learn: Plot classification boundaries with different SVM Kernels. https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html

[8] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30. (SHAP reference for explainability discussion.)